Mobility HW1: Autonomous Driving

Your Name Here

September 2026

1 Kinematic Bicycle Model

1.1 Pepy Model [1 point]

Use a body-fixed longitudinal unit vector $\mathbf{i}_b = (\cos\psi, \sin\psi)$ and lateral unit vector $\mathbf{j}_b = (-\sin\psi, \cos\psi)$, consistent with the lecture's ISO coordinates. Let B be the rear-axle midpoint, F the front-axle midpoint, and G the center of gravity, with $BG = l_r$, $GF = l_f$, and $L = l_f + l_r$. Positive steering and yaw are counterclockwise.

Assumptions

(1) The rigid vehicle moves on a flat plane; roll, pitch, and changes in wheelbase are neglected.

(2) The two front wheels and the two rear wheels are each replaced by one equivalent wheel.

(3) Each wheel rolls along its own orientation with zero lateral tire slip; rear steering is zero.

(4) In the Pepy equations, (X, Y) and speed V refer to the rear-axle midpoint. Steering satisfies $|\delta_f| < \pi/2$.

Rear-wheel rolling implies $\mathbf{v}_B = V\mathbf{i}_b$. For a rigid body with yaw rate $r = \dot{\psi}$,

$$\mathbf{v}_F = \mathbf{v}_B + r\mathbf{k} \times L\mathbf{i}_b = V\mathbf{i}_b + Lr\mathbf{j}_b.$$

The front wheel has a lateral normal $-\sin\delta_f \mathbf{i}_b + \cos\delta_f \mathbf{j}_b$. Zero velocity along that normal gives

$$-V\sin\delta_f + Lr\cos\delta_f = 0 \quad \Rightarrow \quad r = \frac{V}{L}\tan\delta_f.$$

The rear-axle global velocity components now give

$$\boxed{\dot{X} = V\cos\psi, \qquad \dot{Y} = V\sin\psi, \qquad \dot{\psi} = \frac{V}{l_f + l_r}\tan\delta_f.}$$

This derivation uses geometry and rolling constraints, without force or inertia equations, as in Lecture 4.

1.2 Kong Model [1 point]

Retain the assumptions above, but place (X, Y) at G. Denote the rear-axle speed by V_B and the center-of-gravity speed by V. Since $G = B + l_r\mathbf{i}_b$,

$$\mathbf{v}_G = V_B\mathbf{i}_b + l_r r\mathbf{j}_b = V\cos\beta\mathbf{i}_b + V\sin\beta\mathbf{j}_b.$$

Therefore $V_B = V\cos\beta$ and $l_r r = V\sin\beta$. Combining these with $r = V_B\tan\delta_f/L$ yields

$$\tan\beta = \frac{l_r r}{V_B} = \frac{l_r}{L}\tan\delta_f, \qquad \beta = \arctan\left(\frac{l_r}{L}\tan\delta_f\right).$$

Rotating the center-of-gravity velocity into the global frame gives

$$\boxed{\dot{X} = V\cos(\psi + \beta), \qquad \dot{Y} = V\sin(\psi + \beta), \qquad \dot{\psi} = \frac{V}{l_r}\sin\beta.}$$

The Pepy and Kong speeds are attached to different points; generally $V_B = V\cos\beta$, not $V_B = V$.

1.3 Discussion [2 points]

1.3.1 Vehicle Slip Angle - Pepy

The absence of β in the Pepy equations is a consequence of choosing the rear axle as the reference point. With an unsteered rear wheel and ideal rolling, its velocity aligns with the body. The center of gravity still has lateral velocity $l_r\dot{\psi}$, so its vehicle sideslip angle is

$$\boxed{\beta = \arctan\left(\frac{l_r}{l_f + l_r}\tan\delta_f\right).}$$

Thus the center-of-gravity sideslip is generally nonzero while turning, despite zero tire slip.

1.3.2 Vehicle vs. Tire Slip

Vehicle sideslip compares the body axis with the center-of-gravity velocity:

$$\beta = \operatorname{atan2}(v_y, v_x).$$

Tire slip angle compares a wheel's orientation with its local velocity direction. Following Lecture 4, p. 44, use wheel direction minus velocity direction:

$$\alpha_f = \delta_f - \arctan\frac{v_y + l_f r}{v_x}, \qquad \alpha_r = -\arctan\frac{v_y - l_r r}{v_x}.$$

The quantities are related through the vehicle velocities, but need not vanish together. In an ideal kinematic turn, both tire slip angles vanish while β is nonzero. Conversely, with $v_y = 0$ but nonzero yaw rate or steering, $\beta = 0$ while tire slip can be nonzero. In the small-slip dynamic model, the associated lateral axle forces are $F_{yf} = 2C_f\alpha_f$ and $F_{yr} = 2C_r\alpha_r$.

1.4 Implementation [3 points]

I implement the Pepy equations independently using fourth-order Runge–Kutta, starting from $(0, 0, 0)$ with $l_f = l_r = 1.5$ m. Speed is constant in each run and the nominal time step is 0.01 s. Square-wave switches lie on integration boundaries; the final RK4 stage uses the left limit at each switch.

1.4.1 δ equals a nonzero constant

At $V = 5$ m/s, $\delta_f = 0.10, 0.25, 0.45$ rad produces circles with signed radius $R = L/\tan\delta_f$ and curvature $\kappa = \tan\delta_f/L$, so larger positive steering produces a smaller circle. Each simulation lasts 40 s, completing even the largest circle, and agrees with the analytic solution $X = R\sin(Vt/R)$, $Y = R[1 - \cos(Vt/R)]$, $\psi = Vt/R$ within 10^{-7} in the state components.

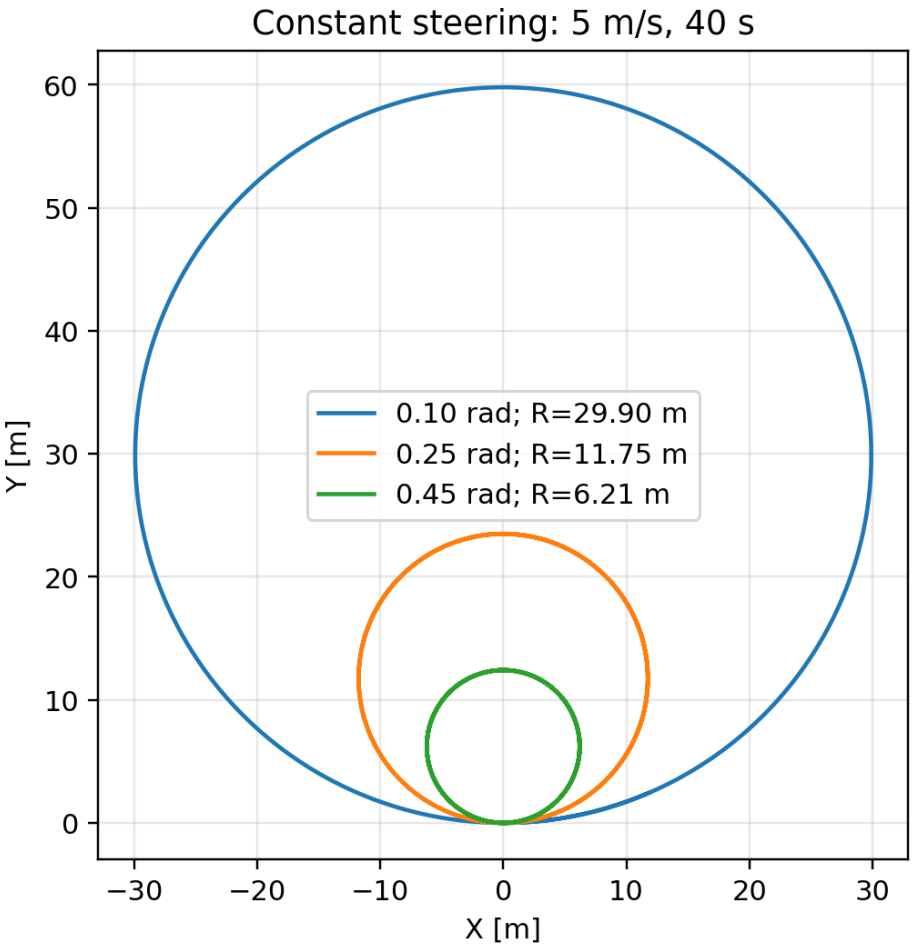


Figure 1: Constant steering trajectories on equal scales.

1.4.2 δ equals $\sin(t)$

For $\delta_f = a \sin t$, I predict a smooth path with alternating curvature but a nonzero average heading: when a is small, $\psi(t) \approx Va(1 - \cos t)/L$, which explains the simulated sideways drift rather than cancellation into a straight line. The comparisons over two periods show that increasing amplitude increases peak curvature, whereas increasing speed increases traveled distance and yaw excursion per temporal period; the larger-amplitude cases also depart further from the small-angle prediction.

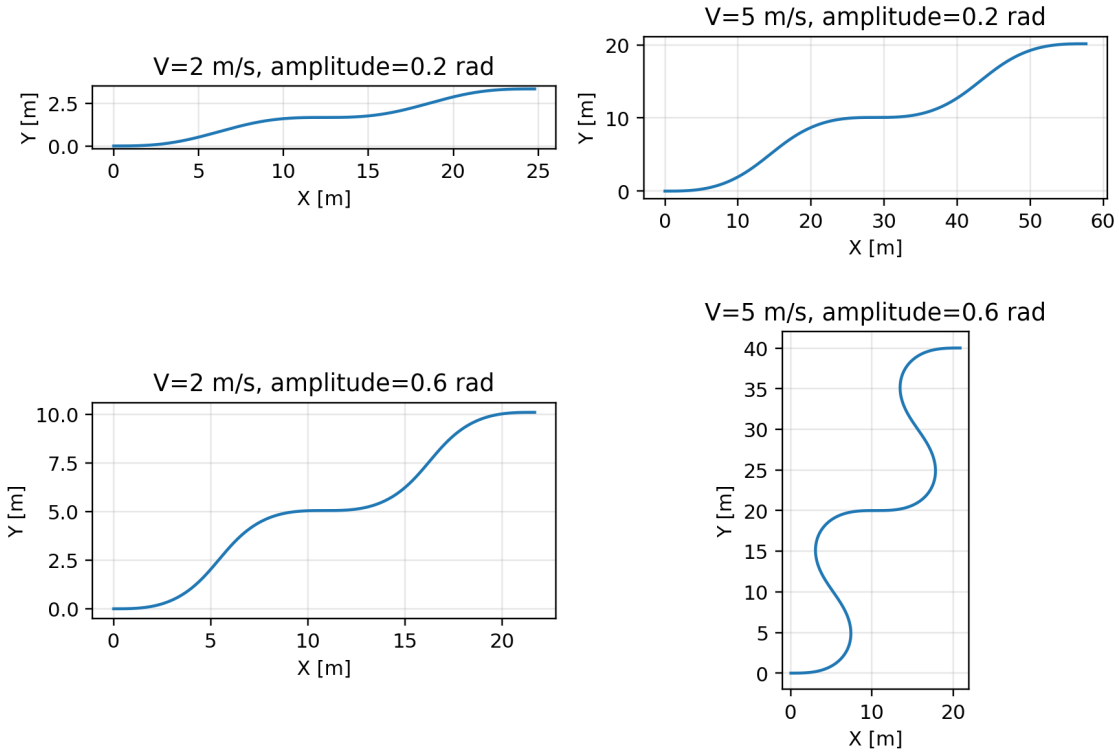


Figure 2: Two speeds and two steering amplitudes; halving the RK4 step changes each terminal state by less than 10^{-6}.

1.4.3 δ equals a square wave

With $V = 4$ m/s and steering alternating between ± 0.25 rad every 2 s, the model generates alternating circular arcs, but instantaneous steering jumps require infinite steering rate and are not physically realizable. To model an actuator, introduce an actual steering state satisfying $\dot{\delta} = \text{clip}((\delta_{\text{cmd}} - \delta)/\tau, -\dot{\delta}_{\max}, \dot{\delta}_{\max})$ and use that continuous state in the vehicle equations.

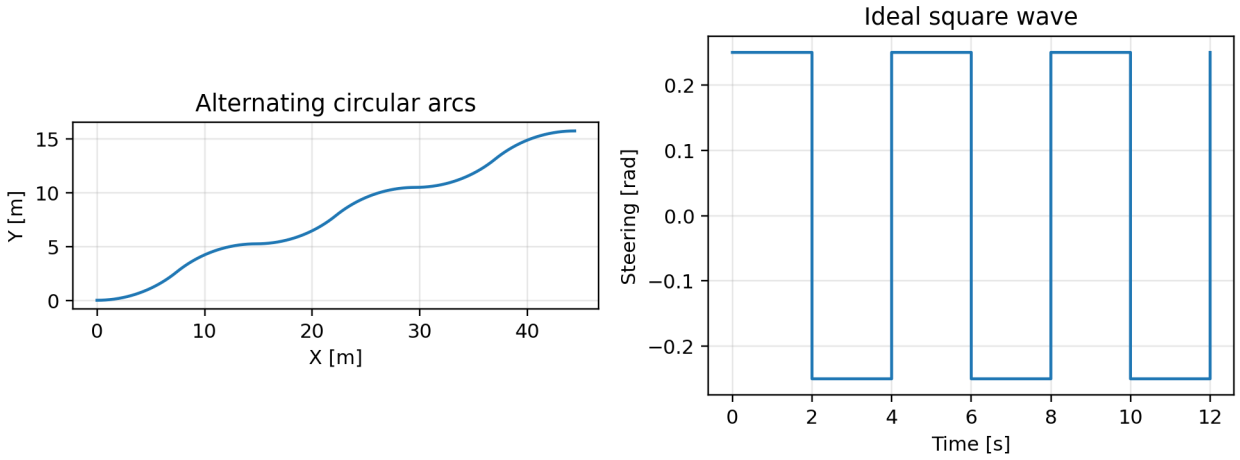


Figure 3: Ideal square-wave command and its trajectory; the actuator extension is proposed rather than included in this plot.

2 Dynamic Bicycle Model

2.1 Model Set Up [0 points]

Use exactly the parameters in the assignment: $V_x = 30$ m/s, $m = 1573$ kg, $I_z = 2873$ kg m^2, $l_f = 1.1$ m, $l_r = 1.58$ m, and $C_f = C_r = 80000$ N/rad. The factors of two account for the two tires per axle as in the lecture derivation. With $x = [e_1, \dot{e}_1, e_2, \dot{e}_2]^T$, define $c = 2(C_f + C_r)$, $d = 2(C_f l_f - C_r l_r)$, and $h = 2(C_f l_f^2 + C_r l_r^2)$. Equation (10) becomes

$$\dot{x} = Ax + B_1\delta + B_2\dot{\psi}_d,$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -c/(mV_x) & c/m & -d/(mV_x) \\ 0 & 0 & 0 & 1 \\ 0 & -d/(I_zV_x) & d/I_z & -h/(I_zV_x) \end{bmatrix}, \quad B_1 = \begin{bmatrix} 0 \\ 2C_f/m \\ 0 \\ 2C_f l_f/I_z \end{bmatrix}, \quad B_2 = \begin{bmatrix} 0 \\ -d/(mV_x) - V_x \\ 0 \\ -h/(I_zV_x) \end{bmatrix}.$$

The controllability matrix has rank four. Lecture 6, pp. 31–37, defines $e_2 = \psi - \psi_d$ and introduces $\dot{e}_1 = v_y + V_x e_2$. Its derivation assumes slowly varying road curvature and neglects $\ddot{\psi}_d$. The implementation retains the supplied matrices and explicitly states the corner-input approximation below.

2.2 Lane Change [2 points]

The desired path is

$$Y_d(X_d) = \begin{cases} -5, & X_d \le 5, \\ -5 + (X_d - 5)/18, & 5 < X_d < 95, \\ 0, & X_d \ge 95. \end{cases}$$

Set $X_d = V_x t$, $\theta = \arctan(5/90) = 0.0554985$ rad, $t_1 = 5/30$ s, and $t_2 = 95/30$ s. Initial error states are zero, corresponding to $(X, Y, \psi) = (0, -5, 0)$. To avoid an undefined continuous-time derivative of the corner heading, use finite-width reference inputs with duration $\Delta t = 0.01$ s and the correct signed heading change:

$$\dot{\psi}_d(t) = \frac{\theta}{\Delta t}\left(\mathbf{1}_{[t_1, t_1+\Delta t)} - \mathbf{1}_{[t_2, t_2+\Delta t)}\right).$$

The integrated reference heading ramps over one time step; it is an approximation to the ideal tangent at each corner. The simulation continues to 6 s, beyond the nominal reference endpoint $(100, 0)$, to measure settling. All input boundaries are handled exactly even when they fall between output samples.

Controller design. I independently search diagonal continuous-time LQR weights and then compute the gain from the algebraic Riccati equation. The chosen design minimizes $J = \int_0^\infty (x^T Q x + \delta^2)\, dt$ with

$$Q = \mathrm{diag}(0.1811228190,\ 0.0003019885,\ 5.5884459878,\ 0.1870022338), \quad R = 1.$$

Solving $A^T P + PA - PB_1 B_1^T P + Q = 0$ and setting $K = B_1^T P$ gives

$$\boxed{\delta = -Kx, \qquad K = [0.42558527,\ 0.05773618,\ 3.27261580,\ 0.38541735].}$$

The closed-loop poles are

$$\boxed{-25.12034674, \quad -10.38059735, \quad -3.82198513 \pm 3.18895699i.}$$

They all have negative real parts. No feedforward input, state reset, or steering saturation is added.

Numerical verification. The primary simulator uses exact matrix-exponential propagation for each constant-input interval, returning samples every 0.01 s. A separately implemented segmented adaptive ODE integrator agrees within 1.71×10^{-12} in the state components. A 0.0001 s verification grid, including switching instants and one-second deadlines, yields:

Test in the supplied error model	Result	Required bound
Peak $\lvert e_2 \rvert$	0.00952359 rad	0.01 rad
First transition settling	0.99733 s	1 s
Second transition settling	0.99733 s	1 s
Peak continuous $\lvert \dot{\delta} \rvert$	23.80862 rad/s	25 rad/s
Peak sampled $\lvert \Delta\delta \rvert / \Delta t$	11.64185 rad/s	25 rad/s

Settling means both $|e_1| \le 0.002$ m and $|e_2| \le 0.0007$ rad hold and remain satisfied until the next transition or simulation end. The maximum transient $|e_1|$ is 0.09204 m; no all-time lateral-error bound is specified. Steering rate is evaluated from $\dot{\delta} = -K[(A - B_1 K)x + B_2 \dot{\psi}_d]$, checking both one-sided derivatives at input switches as well as finite differences of the saved steering samples.

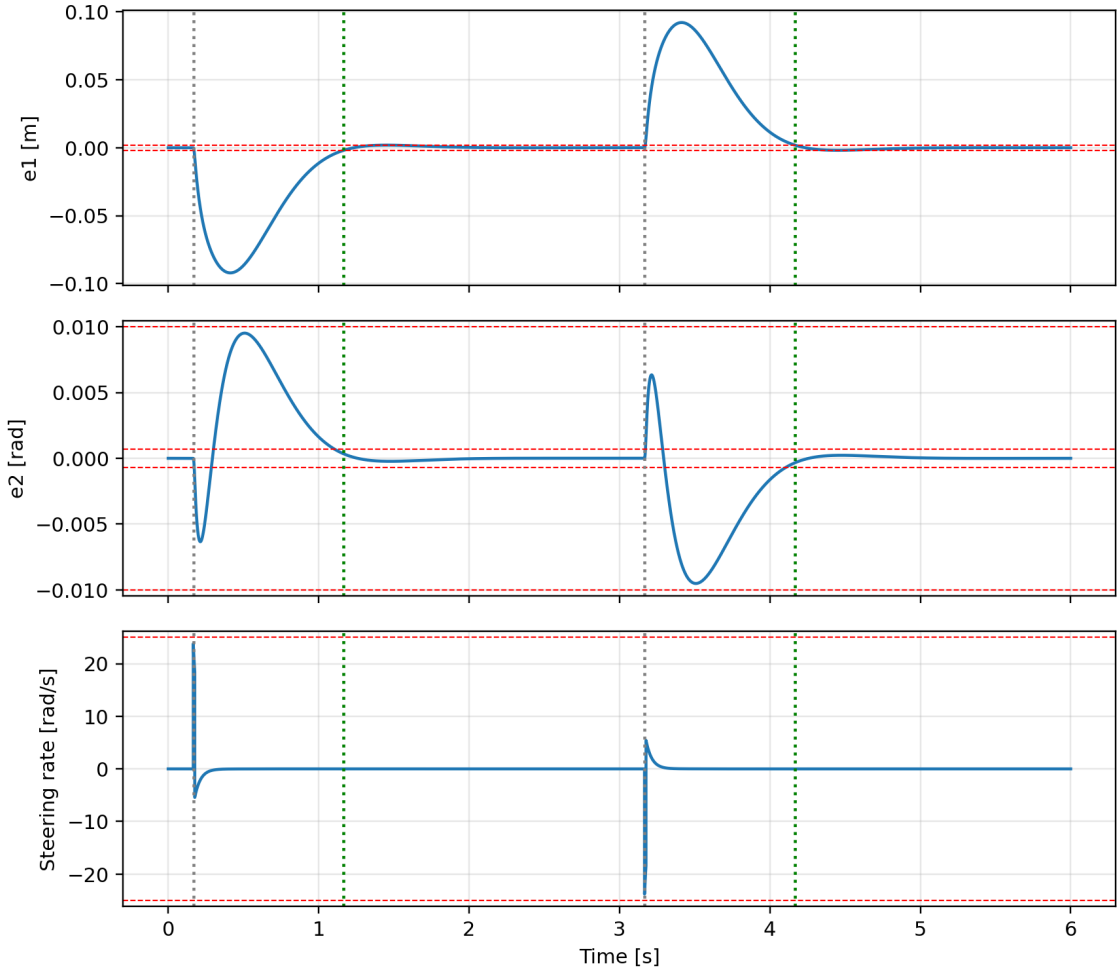


Figure 4: Course-model errors and steering rate. Gray lines mark transitions, green lines mark one-second deadlines, and red lines show the bounds.

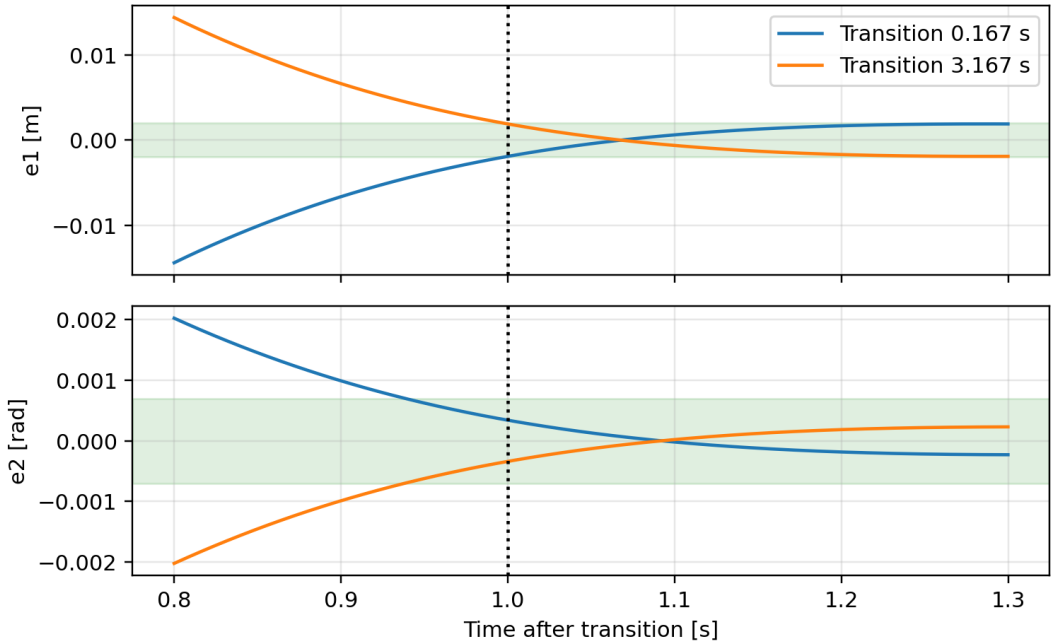


Figure 5: Both transitions aligned by elapsed time; green bands indicate simultaneous settling tolerances.

2.3 Desired and Actual Plot [1.5 points]

Following Lecture 6, p. 31, use the body-lateral geometric relation with positive e_1 toward the vehicle-left side of the reference point:

$$\psi = \psi_d + e_2, \qquad X = X_d - e_1 \sin\psi, \qquad Y = Y_d + e_1 \cos\psi.$$

This reconstructs global coordinates within the course's error-model convention. Projecting the displacement back onto the body lateral axis recovers e_1 with maximum difference 4.52×10^{-16} m. This checks the coordinate conversion, not independent physical feasibility of the corner approximation. The figures use unequal axis scales to make the small lateral deviations visible.

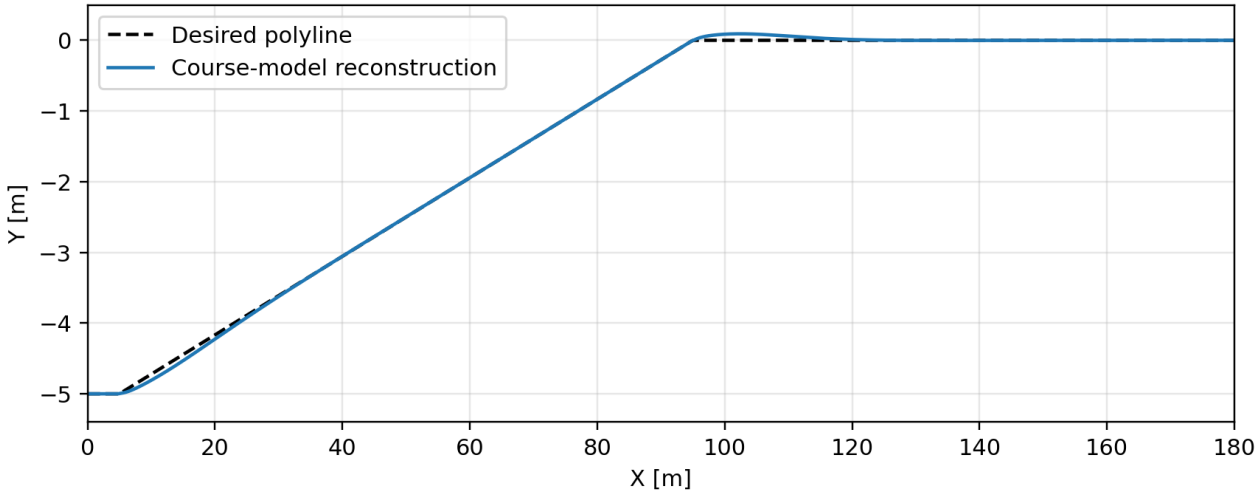


Figure 6: Desired path and reconstructed global course-model trajectory.

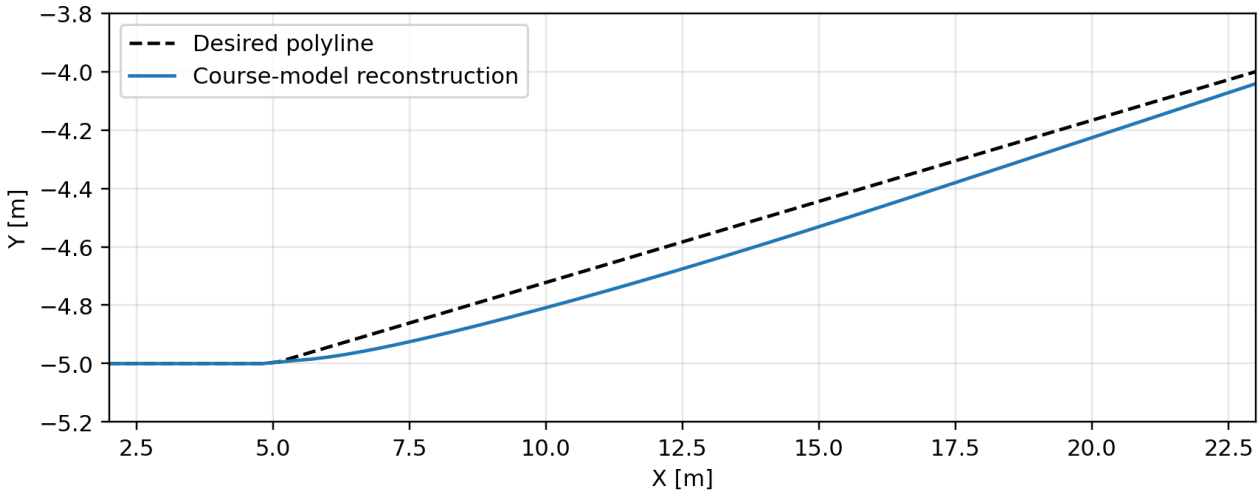


Figure 7: Separate zoom at departure from the original lane.

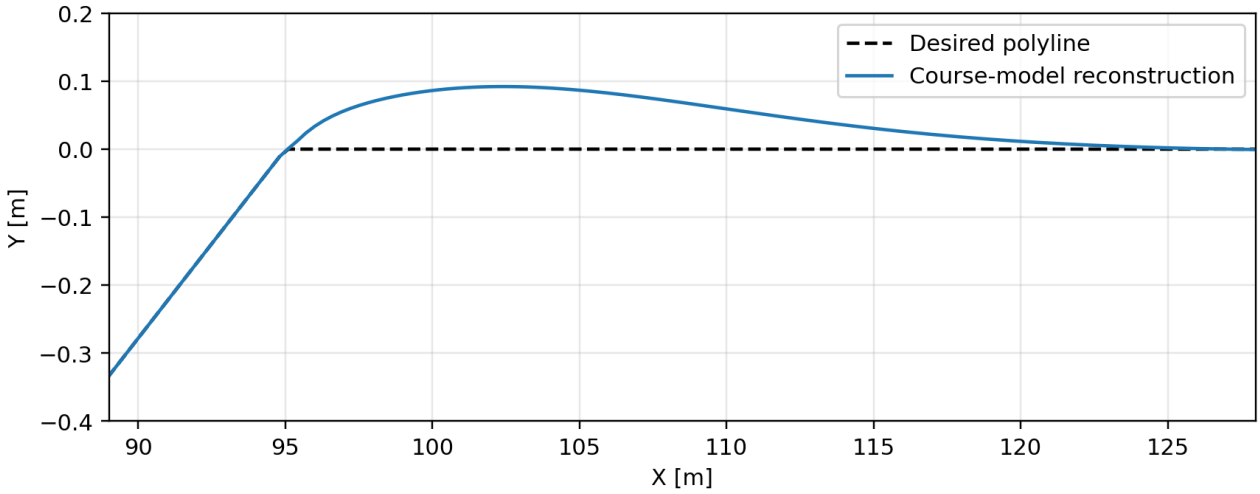


Figure 8: Separate zoom at entry into the new lane.

Endpoint and reference audit. The error-model results above pass the stated numerical peak/rate/settling tests for the declared input representation. They do not prove every literal geometric condition. At nominal $X_d = 100$ m, the reconstructed position is approximately $(100.000307, 0.086239)$ m and heading is -0.003557 rad; thus this candidate has not reached $(100, 0)$ with zero heading there. By 6 s the reconstructed lateral position is 4.875×10^{-6} m, consistent with eventual settling.

Moreover, a finite-time reference heading ramp differs from the ideal polyline tangent. Recomputing heading error against that ideal tangent gives a peak of approximately 0.055499 rad, rather than the smaller internal e_2. The script reports these as separate tests. For a continuous physical heading and an instantaneous reference jump of θ, the larger of the two one-sided heading errors is at least $\theta/2 = 0.0277493$ rad, so the strict all-time 0.01 rad condition cannot hold under that particular interpretation.

Consequently, this is a verified candidate for the supplied approximate error-model exercise, not a claim that the literal sharp-corner and finite-endpoint conditions are all satisfied. The provided lecture derivation does not specify a grading convention for these assignment corner transitions. The archive includes a separate independently written discrete pole search with the endpoint constraint; it did not find a feasible candidate in its tested family, which is not a general infeasibility proof.

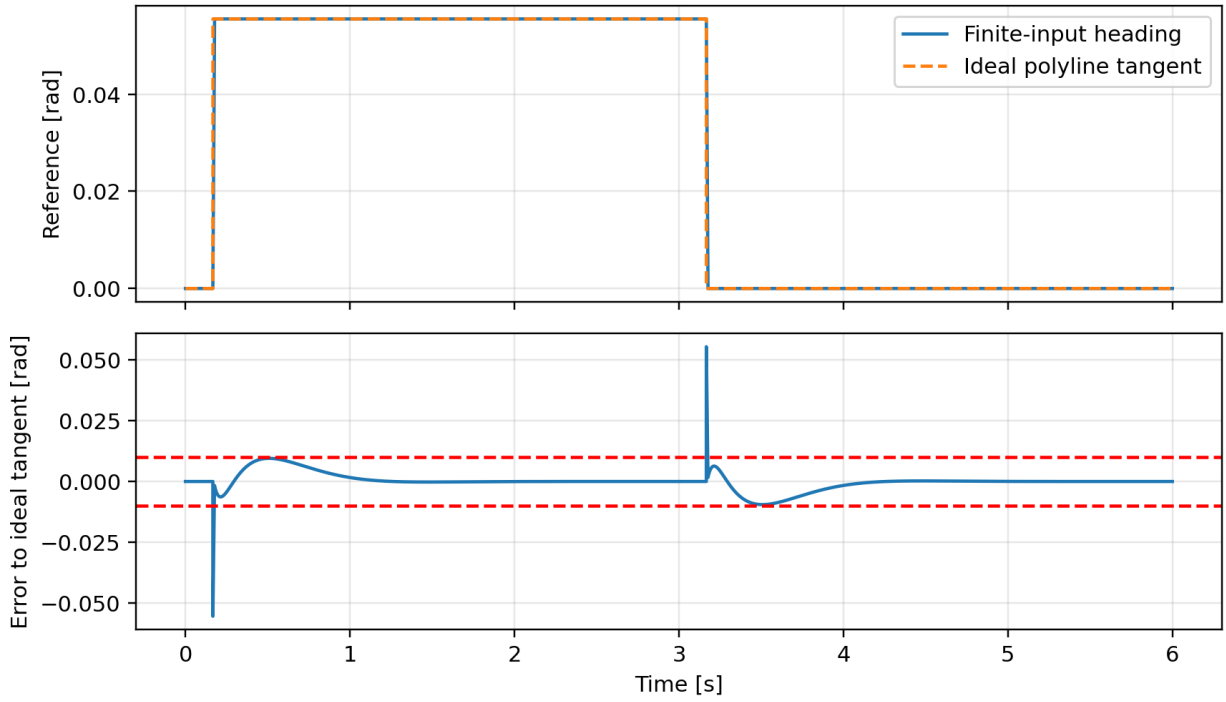


Figure 9: The finite-input heading and ideal tangent agree away from corners; the lower panel audits heading error against the ideal tangent.

Sources. The supplied *Autonomous Driving Mobility Assignment* (September 2026), Equation (10); J. M. Dolan, *AD Lecture 4: Modeling* (Fall 2026), kinematic derivation and p. 44 tire-slip convention; *AD Lecture 5: Controls 1*, pp. 51–57, LQR; *AD Lecture 6: Controls 2*, pp. 31–37 and 43–44, error definitions, approximations, and control design.

3 Appendix

All implementation below is new and self-contained. Run `python run.py`; the supplementary search scripts are optional and are included in the archive. The fixed design weights used for the reported run are stored in `design.json`.

3.1 1.4.A Pepy Model constant turning implementation

Shared models and LQR solver: `bicycle.py`

```
"""Models transcribed from the assignment and Lectures 4 and 6."""
from dataclasses import dataclass
import numpy as np
from scipy.linalg import solve_continuous_are


@dataclass(frozen=True)
class Vehicle:
    speed: float = 30.0
    mass: float = 1573.0
    inertia: float = 2873.0
    front: float = 1.1
    rear: float = 1.58
    cf: float = 80000.0
    cr: float = 80000.0

    def error_matrices(self):
        v, m, j = self.speed, self.mass, self.inertia
        a, b, f, r = self.front, self.rear, self.cf, self.cr
        A = np.array([
            [0, 1, 0, 0],
            [0, -2*(f+r)/(m*v), 2*(f+r)/m, -2*(f*a-r*b)/(m*v)],
            [0, 0, 0, 1],
            [0, -2*(f*a-r*b)/(j*v), 2*(f*a-r*b)/j,
             -2*(f*a*a+r*b*b)/(j*v)]])
        B = np.array([0, 2*f/m, 0, 2*f*a/j])
        D = np.array([0, -2*(f*a-r*b)/(m*v)-v, 0,
                      -2*(f*a*a+r*b*b)/(j*v)])
        return A, B, D


def lqr(vehicle, diagonal):
    A, B, D = vehicle.error_matrices()
    Q = np.diag(diagonal)
    P = solve_continuous_are(A, B[:, None], Q, np.ones((1, 1)))
    K = B@P
    residual = A.T@P+P@A-np.outer(P@B, B@P)+Q
    assert np.max(abs(residual)) < 1e-8
    controllability = np.column_stack([np.linalg.matrix_power(A, k)@B
                                       for k in range(4)])
    assert np.linalg.matrix_rank(controllability) == 4
    return A-np.outer(B, K), D, K


def pepy(state, velocity, steering, wheelbase=3.):
    yaw = state[2]
    return np.array([velocity*np.cos(yaw), velocity*np.sin(yaw),
                     velocity*np.tan(steering)/wheelbase])
```

RK4, all three steering experiments, checks and plotting: `kinematics.py`

```
"""Core 1.4: RK4 simulations, figures, and analytic/refinement checks."""
from pathlib import Path
import json
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from bicycle import pepy

ROOT = Path(__file__).resolve().parent
FIG, DATA = ROOT/'figures', ROOT/'data'
FIG.mkdir(exist_ok=True)
DATA.mkdir(exist_ok=True)


def rk4(speed, steering, stop, step=.01):
```

```
    times = np.r_[np.arange(0, stop, step), stop]
    states = np.zeros((len(times), 3))
    for k, (t0, t1) in enumerate(zip(times[:-1], times[1:])):
        h, z = t1-t0, states[k]
        k1 = pepy(z, speed, steering(t0))
        k2 = pepy(z+h*k1/2, speed, steering(t0+h/2))
        k3 = pepy(z+h*k2/2, speed, steering(t0+h/2))
        k4 = pepy(z+h*k3, speed, steering(np.nextafter(t1, t0)))
        states[k+1] = z+h*(k1+2*k2+2*k3+k4)/6
    return times, states


def figure_save(fig, filename):
    fig.tight_layout()
    fig.savefig(FIG/filename, dpi=220, bbox_inches='tight')
    plt.close(fig)


def export(name, t, z):
    np.savetxt(DATA/name, np.column_stack([t, z]), delimiter=',',
               header='time_s,X_m,Y_m,yaw_rad', comments='')


def constant():
    fig, ax = plt.subplots(figsize=(6.5, 5))
    checks = {}
    for angle in [.10, .25, .45]:
        t, z = rk4(5., lambda time: angle, 40.)
        radius = 3/np.tan(angle)
        yaw = 5*t/radius
        truth = np.column_stack([radius*np.sin(yaw),
                                 radius*(1-np.cos(yaw)), yaw])
        error = float(np.max(abs(z-truth)))
        assert error < 1e-7
        checks[str(angle)] = dict(radius_m=float(radius),
                                  revolution_s=float(2*np.pi*radius/5),
                                  max_state_difference=error)
        ax.plot(z[:, 0], z[:, 1], label=f'{angle:.2f} rad; R={radius:.2f} m')
        export(f'constant_{angle}.csv', t, z)
    ax.set(xlabel='X [m]', ylabel='Y [m]', title='Constant steering: 5 m/s, 40 s')
    ax.set_aspect('equal', adjustable='box'); ax.legend(); ax.grid(alpha=.3)
    figure_save(fig, 'constant.png')
    return checks


def sinusoidal():
    fig, axes = plt.subplots(2, 2, figsize=(8, 6))
    checks = {}
    for ax, (speed, amp) in zip(axes.flat, [(2,.2), (5,.2), (2,.6), (5,.6)]):
        control = lambda time: amp*np.sin(time)
        t, z = rk4(speed, control, 4*np.pi)
        _, fine = rk4(speed, control, 4*np.pi, .005)
        discrepancy = float(np.max(abs(z[-1]-fine[-1])))
        assert discrepancy < 1e-6
        checks[f'V{speed}_a{amp}'] = discrepancy
        ax.plot(z[:, 0], z[:, 1])
        ax.set(title=f'V={speed} m/s, amplitude={amp} rad',
               xlabel='X [m]', ylabel='Y [m]')
        ax.set_aspect('equal', adjustable='box'); ax.grid(alpha=.3)
        export(f'sine_V{speed}_a{amp}.csv', t, z)
    figure_save(fig, 'sinusoidal.png')
    return checks


def square():
    def control(time):
        return .25 if int(np.floor(time/2)) % 2 == 0 else -.25
    t, z = rk4(4., control, 12.)
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))
    axes[0].plot(z[:, 0], z[:, 1])
    axes[0].set(xlabel='X [m]', ylabel='Y [m]', title='Alternating circular arcs')
    axes[0].set_aspect('equal', adjustable='box')
    axes[1].step(t, [control(time) for time in t], where='post')
    axes[1].set(xlabel='Time [s]', ylabel='Steering [rad]', title='Ideal square wave')
    for ax in axes: ax.grid(alpha=.3)
    figure_save(fig, 'square.png')
    export('square.csv', t, z)


if __name__ == '__main__':
    checks = dict(constant=constant(), sinusoid_refinement=sinusoidal())
    square()
    (DATA/'kinematic_checks.json').write_text(json.dumps(checks, indent=2))
```

```
    print('Kinematic checks passed.')
```

3.2 1.4.B Pepy Model sinusoidal turning implementation

The complete implementation is the `sinusoidal()` function in the preceding listing. It calls the same RK4 integrator with two step sizes for an independent refinement check.

3.3 1.4.C Pepy Model square wave turning implementation

The complete implementation is the `square()` function in the preceding listing. The RK4 endpoint convention treats the left and right limits of the ideal steering jumps consistently.

3.4 2.1 Dynamic Bicycle Model Lane Change Setup

```
"""Core 2: independent implementation of assignment Equation (10).
Primary simulation: exact constant-input flow, 0.01-s output.
Independent verification: segmented solve_ivp on a finer time grid.
"""
import json
import numpy as np
from scipy.linalg import expm
from scipy.integrate import solve_ivp
from bicycle import Vehicle, lqr
from kinematics import ROOT, DATA, figure_save, plt

STEP, STOP = .01, 6.
VEHICLE = Vehicle()
EVENTS = np.array([5., 95.])/VEHICLE.speed
ANGLE = np.arctan2(5., 90.)
BOUNDS = np.r_[0., EVENTS[0], EVENTS[0]+STEP,
               EVENTS[1], EVENTS[1]+STEP, STOP]
INPUTS = np.array([0., ANGLE/STEP, 0., -ANGLE/STEP, 0.])


def reference(t):
    t = np.asarray(t)
    X = VEHICLE.speed*t
    Y = np.clip((X-5)/18, 0, 5)-5
    heading = ANGLE*(np.clip((t-EVENTS[0])/STEP, 0, 1)
                     -np.clip((t-EVENTS[1])/STEP, 0, 1))
    rate = np.zeros_like(t)
    for left, right, value in zip(BOUNDS[:-1], BOUNDS[1:], INPUTS):
        rate[(t >= left) & (t < right)] = value
    return X, Y, heading, rate


def exact_samples(F, D, samples):
    # Merge the output times with switching times, so no step straddles a jump.
    merged = np.unique(np.r_[samples, BOUNDS])
    states = np.zeros((len(merged), 4))
    augmented = np.zeros((5, 5))
    augmented[:4, :4], augmented[:4, 4] = F, D
    flows = {}
    for j in range(len(merged)-1):
        left, right = merged[j:j+2]
        duration = round(float(right-left), 14)
        if duration not in flows:
            flows[duration] = expm(augmented*duration)
        E = flows[duration]
        segment = np.searchsorted(BOUNDS, (left+right)/2, side='right')-1
        states[j+1] = E[:4, :4]@states[j]+E[:4, 4]*INPUTS[segment]
    return states[np.searchsorted(merged, samples)]


def independent_ode(F, D, times):
    result = np.zeros((len(times), 4))
    initial = np.zeros(4)
    for left, right, value in zip(BOUNDS[:-1], BOUNDS[1:], INPUTS):
        sol = solve_ivp(lambda time, z: F@z+D*value,
                        (left, right), initial, dense_output=True,
                        rtol=2e-11, atol=2e-13, max_step=.002)
        assert sol.success
```

```
        chosen = (times >= left) & (times <= right)
        result[chosen] = sol.sol(times[chosen]).T
        initial = sol.y[:, -1]
    return result


def reconstruct(t, states):
    Xd, Yd, psid, _ = reference(t)
    psi = psid+states[:, 2]
    # Lecture 6 p31; positive e1 denotes vehicle-left from reference point.
    X = Xd-states[:, 0]*np.sin(psi)
    Y = Yd+states[:, 0]*np.cos(psi)
    recovered = -(X-Xd)*np.sin(psi)+(Y-Yd)*np.cos(psi)
    return X, Y, psi, recovered


def main():
    weights = json.loads((ROOT/'design.json').read_text())
    F, D, K = lqr(VEHICLE, weights['Q_diagonal'])
    poles = np.linalg.eigvals(F)
    assert np.all(poles.real < 0)
    t = np.arange(round(STOP/STEP)+1)*STEP
    x = exact_samples(F, D, t)
    dense_t = np.unique(np.r_[np.linspace(0, STOP, 60001), BOUNDS, EVENTS+1, 100/30])
    dense_x = independent_ode(F, D, dense_t)
    independent_at_samples = independent_ode(F, D, t)
    solver_error = float(np.max(abs(x-independent_at_samples)))
    assert solver_error < 1e-8
    _, _, _, rd = reference(dense_t)
    steer = -x@K
    steer_rate = -(dense_x@F.T+rd[:, None]*D)@K
    # Add both limits at each switch; steering is continuous in this model.
    endpoint_rates = []
    xb = exact_samples(F, D, BOUNDS)
    for j in range(1, len(BOUNDS)-1):
        endpoint_rates.extend([abs(K@(F@xb[j]+D*u))
                               for u in INPUTS[j-1:j+1]])
    rate_peak = float(max(np.max(abs(steer_rate)), *endpoint_rates))
    settling = []
    for start, end in zip(EVENTS, [EVENTS[1], STOP]):
        in_segment = (dense_t >= start) & (dense_t < end)
        outside = (abs(dense_x[:, 0]) > .002) | (abs(dense_x[:, 2]) > .0007)
        failures = np.flatnonzero(in_segment & outside)
        settling.append(float(dense_t[failures[-1]+1]-start) if failures.size else 0.)
    X, Y, psi, recovered = reconstruct(t, x)
    Xd, Yd, psid, _ = reference(t)
    geometry_error = float(np.max(abs(recovered-x[:, 0])))
    assert geometry_error < 1e-12
    terminal_t = np.array([100/30])
    terminal_x = exact_samples(F, D, terminal_t)
    terminal = reconstruct(terminal_t, terminal_x)
    sharp_heading = np.where((dense_t >= EVENTS[0]) & (dense_t < EVENTS[1]), ANGLE, 0.)
    finite_heading = reference(dense_t)[2]+dense_x[:, 2]
    diagnostics = dict(
        Q=weights['Q_diagonal'], R=1, K=K.tolist(),
        closed_loop_poles=[dict(real=float(z.real), imag=float(z.imag)) for z in poles],
        sample_interval_s=STEP, verification_interval_s=.0001,
        solver_max_difference=solver_error, geometry_max_difference=geometry_error,
        max_e1_m=float(np.max(abs(dense_x[:, 0]))),
        max_e2_rad=float(np.max(abs(dense_x[:, 2]))),
        max_continuous_steering_rate_rad_s=rate_peak,
        max_sampled_steering_rate_rad_s=float(np.max(abs(np.diff(steer)/STEP))),
        settling_s=settling,
        at_nominal_X100=dict(X=float(terminal[0][0]), Y=float(terminal[1][0]),
                             heading=float(terminal[2][0])),
        at_simulation_end=dict(X=float(X[-1]), Y=float(Y[-1]), heading=float(psi[-1])),
        max_heading_error_against_ideal_tangent_rad=float(np.max(abs(finite_heading-sharp_heading))),
        continuous_heading_corner_lower_bound_rad=float(ANGLE/2))
    diagnostics['course_model_checks'] = dict(
        heading=diagnostics['max_e2_rad'] <= .01,
        steering_rate=rate_peak <= 25,
        settling=max(settling) <= 1,
        initial_state=bool(np.all(x[0] == 0)))
    assert all(diagnostics['course_model_checks'].values())
    diagnostics['literal_checks'] = dict(
        ideal_tangent_heading=diagnostics['max_heading_error_against_ideal_tangent_rad'] <= .01,
        X100_within_error_bands=bool(abs(terminal_x[0, 0]) <= .002 and
                                     abs(terminal_x[0, 2]) <= .0007))
    (DATA/'lane_validation.json').write_text(json.dumps(diagnostics, indent=2))
    np.savetxt(DATA/'lane_samples.csv',
               np.column_stack([t, x, steer, Xd, Yd, X, Y, psi]), delimiter=',',
               header='t,e1,e1dot,e2,e2dot,delta,X_des,Y_des,X,Y,psi', comments='')
    fig, axes = plt.subplots(3, 1, figsize=(8, 7), sharex=True)
```

```
    for ax, values, label, limits in zip(axes,
            [dense_x[:, 0], dense_x[:, 2], steer_rate],
            ['e1 [m]', 'e2 [rad]', 'Steering rate [rad/s]'],
            [[.002], [.0007,.01], [25]]):
        ax.plot(dense_t, values)
        for threshold in limits:
            ax.axhline(threshold, color='red', ls='--', lw=.7)
            ax.axhline(-threshold, color='red', ls='--', lw=.7)
        for event in EVENTS:
            ax.axvline(event, color='gray', ls=':')
            ax.axvline(event+1, color='green', ls=':')
        ax.set_ylabel(label); ax.grid(alpha=.3)
    axes[-1].set_xlabel('Time [s]')
    figure_save(fig, 'lane_errors.png')
    fig, axes = plt.subplots(2, 1, figsize=(7, 4.4), sharex=True)
    for ax, col, limit in zip(axes, [0,2], [.002,.0007]):
        for event in EVENTS:
            sel=(dense_t>=event+.8)&(dense_t<=event+1.3)
            ax.plot(dense_t[sel]-event,dense_x[sel,col],label=f'Transition {event:.3f} s')
        ax.axhspan(-limit,limit,color='green',alpha=.12)
        ax.axvline(1,color='black',ls=':'); ax.grid(alpha=.3)
        ax.set_ylabel('e1 [m]' if col==0 else 'e2 [rad]')
    axes[0].legend(); axes[-1].set_xlabel('Time after transition [s]')
    figure_save(fig,'lane_settling.png')
    for filename, xr, yr in [('lane_global.png',(0,180),(-5.4,.5)),
                             ('lane_start.png',(2,23),(-5.2,-3.8)),
                             ('lane_end.png',(89,128),(-.4,.2))]:
        fig, ax = plt.subplots(figsize=(8, 3.3))
        ax.plot(Xd, Yd, 'k--', label='Desired polyline')
        ax.plot(X, Y, label='Course-model reconstruction')
        ax.set(xlim=xr, ylim=yr, xlabel='X [m]', ylabel='Y [m]')
        ax.legend(); ax.grid(alpha=.3)
        figure_save(fig, filename)
    fig, axes = plt.subplots(2,1,figsize=(8,4.7),sharex=True)
    axes[0].plot(dense_t, reference(dense_t)[2],label='Finite-input heading')
    axes[0].plot(dense_t,sharp_heading,'--',label='Ideal polyline tangent')
    axes[0].set_ylabel('Reference [rad]'); axes[0].legend()
    axes[1].plot(dense_t,finite_heading-sharp_heading)
    axes[1].axhline(.01,color='red',ls='--'); axes[1].axhline(-.01,color='red',ls='--')
    axes[1].set(xlabel='Time [s]',ylabel='Error to ideal tangent [rad]')
    for ax in axes:ax.grid(alpha=.3)
    figure_save(fig,'reference_audit.png')
    print(json.dumps(diagnostics, indent=2))


if __name__ == '__main__':
    main()
```

Complete numerical design input: `design.json`

```
{
  "Q_diagonal": [
    0.1811228190184231,
    0.0003019884889917931,
    5.588445987846785,
    0.18700223381189277
  ],
  "R": 1,
  "search_seed": 642
}
```

Reproduction entry point: `run.py`

```
"""Run the newly implemented Core simulations from any working directory."""
from pathlib import Path
import subprocess
import sys

root = Path(__file__).resolve().parent
for name in ['kinematics.py', 'lane_change.py']:
    subprocess.run([sys.executable, str(root/name)], cwd=root, check=True)
print('All implemented numerical checks completed; read lane_validation.json for scope.')
```